CAST Technical Specification

# CAST PCI-M64AHB Core: 64-bit/66MHz PCI to AMBA AHB Interface

## 1 Introduction

The main purpose of the PCI-M64AHB Core is to act as a simple bridge between the PCI bus and the AMBA AHB bus.

The PCI-M64AHB Interface supports a 64-bit address/data bus and operates at up to a 66MHz PCI clock frequency. It is fully compliant with the PCI Local Bus Specification, Revision 2.2. The PCI-M64AHB core can be used in ASIC and FPGA devices.

The interface implements 64 bytes of PCI Configuration Space registers. It is possible to extend the Configuration Space up to 256 bytes for application needs.

The core allows PCI-AHB transfers and DMA data transfers in both directions.

The PCI-AHB transfer window can range from 16 bytes to 2GB.

## 2 Features

- PCI specification 2.2 compliant
- 66MHz PCI performance
- 64-bit PCI data path
- Zero wait states burst mode
- Full PCI bus master/target functionality
- Single PCI interrupt support
- Type 0 PCI Configuration space
- PCI Parity generation and parity error detection.
- PCI-AHB window size from 16B to 2GB
- 64-bit DMA Controller
- Eight 64-bit Mailboxes
- AHB bus interrupt
- Available in synthesizable HDL source code
- Verified in Xilinx Virtex-II FPGA

# 3 Interfaces

## 3.1 Hardware Interface

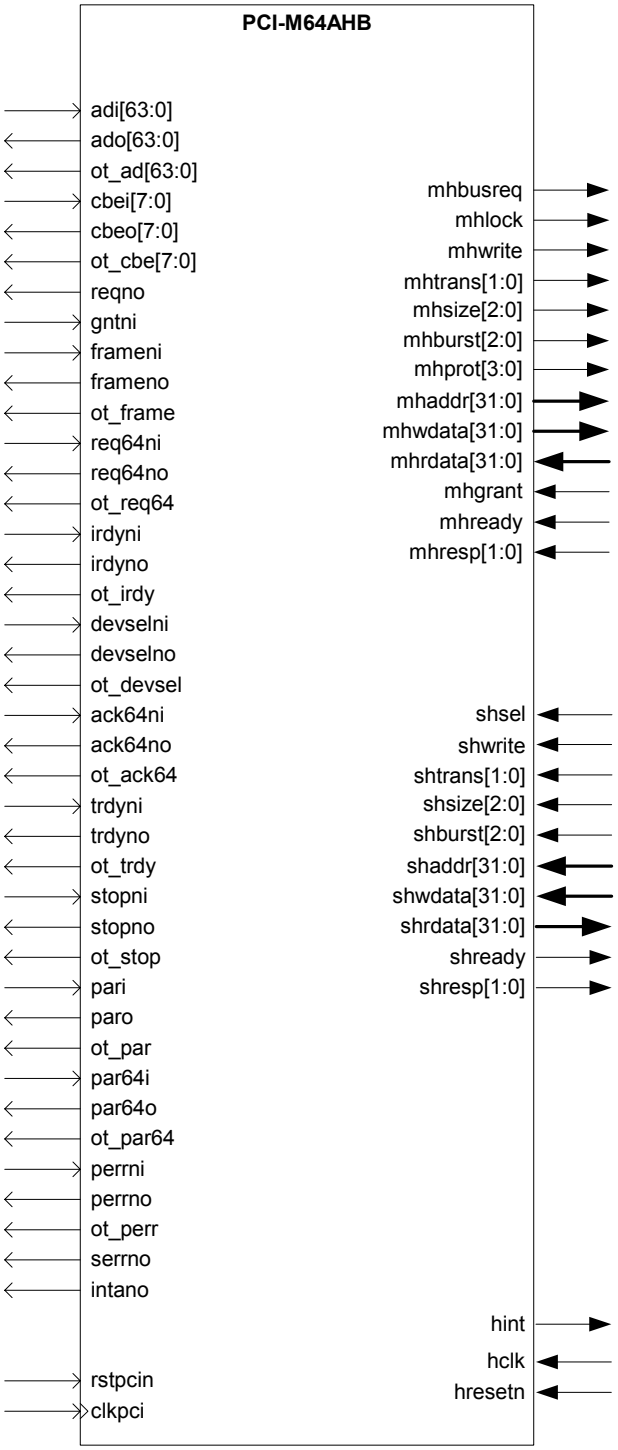


**Figure 1 - PCI-M64AHB Pinout**

| Name | Type | Polarity | Description |
|---|---|---|---|
| **PCI Interface** | | | |
| clkpci | In | - | PCI system clock |
| rstpcin | In | Low | System Reset |
| adi[63:0] | In | - | Address/Data bus input |
| ado[63:0] | Out | - | Address/Data bus output |
| ot_ad [63:0] | Out | - | Address/Data bus output tri-state control |
| cbei[7:0] | In | - | Command/Byte Enable bus input |
| cbeo[7:0] | Out | - | Command/Byte Enable bus output |
| ot_cbe [7:0] | Out | - | Command/Byte Enable bus output tri-state control |
| pari | In | High | 32-bit parity input. |
| paro | Out | High | 32-bit parity output. |
| ot_par | Out | High | 32-bit parity output tri-state control. |
| par64i | In | High | 64-bit parity input |
| par64o | Out | High | 64-bit parity output. |
| ot_par64 | Out | High | 64-bit parity output tri-state control. |
| perrni | In | Low | Parity error input |
| perrno | Out | Low | Parity error output |
| ot_perr | Out | High | Parity error output tri-state control. |
| serrni | Out | Low | System Error input |
| idseli | In | High | Initialization device select input |
| frameni | In | Low | Transaction Frame input. |
| frameno | Out | Low | Transaction Frame output |
| ot_frame | Out | High | Transaction Frame output tri-state control. |
| req64ni | In | Low | 64-bit Transaction Request input. |
| req64no | Out | Low | 64-bit Transaction Request output. |
| ot_req64 | Out | High | 64-bit Transaction Request output tri-state control. |
| irdyni | In | Low | Initiator ready input. |
| irdyno | Out | Low | Initiator ready output. |
| ot_irdy | Out | High | Initiator ready output tri-state control. |
| devselni | In | Low | Device select input. |
| devselno | Out | Low | Device select output. |
| ot_devsel | Out | High | Device select output tri-state control. |
| ack64ni | In | Low | 64-bit Transaction Acknowledge input. |
| ack64no | Out | Low | 64-bit Transaction Acknowledge output. |
| ot_ack64 | Out | High | 64-bit Transaction Acknowledge output tri-state control. |
| trdyni | In | Low | Target Ready input. |
| trdyno | Out | Low | Target Ready output. |
| ot_trdy | Out | High | Target Ready output tri-state control. |
| stopni | In | Low | Stop input. |
| stopni | Out | Low | Stop output. |
| ot_stop | Out | High | Stop output tri-state control. |
| reqno | Out | Low | PCI Bus request |
| gntni | In | Low | PCI Bus grant input. |
| intano | Out | Low | Interrupt pin output. |
| **AMBA AHB Master Interface** | | | |
| hclk | In | - | AHB Clock |
| hresetn | In | Low | AHB bus reset |
| mhreq | Out | High | AHB bus mastership request |
| mhgrant | In | High | AHB bus mastership grant |
| mhtrans[1:0] | Out | - | Transfer Type |
| mhsize[2:0] | Out | - | Transfer Size |
| mhburst[2:0] | Out | - | Burst Type |
| mhprot[3:0] | Out | - | Protection |
| mhaddr[31:0] | Out | - | AHB Address |
| mhresp[1:0] | In | - | Transfer response |

| Name | Type | Polarity | Description |
|---|---|---|---|
| mhready | In | High | Transfer Done |
| mhwdata[31:0] | Out | - | Write Data bus |
| mhrdata[31:0] | In | - | Read Data bus |
| **AMBA AHB Slave Interface** | | | |
| shsel | In | High | Slave select |
| shtrans[1:0] | In | - | Transfer Type |
| ssize[2:0] | In | - | Transfer Size |
| sburst[2:0] | In | - | Burst Type |
| shprot[3:0] | In | - | Protection |
| shaddr[31:0] | In | - | AHB Address |
| shresp[1:0] | Out | - | Transfer response |
| shready | Out | High | Transfer Done |
| shwdata[31:0] | In | - | Write Data bus |
| shrdata[31:0] | Out | - | Read Data bus |
| hint | Out | High | AHB Interrupt |

**Table 1 – Signal Descriptions**

## 3.2 PCI Configuration Space

Each logical PCI bus device includes a block of 64 configuration DWORDS reserved for the implementation of its configuration registers. The format of the first 16 DWORDS is defined by the PCI Special Interest Group (PCI SIG) in the PCI Local Bus Specification, Revision 2.2. This specification defines two header formats, type one and type zero. Header type one is used for PCI-to-PCI bridges. Header type zero is used for all other devices. The PCI-M64AHB Core supports only header type zero.

Table 2 shows the defined 64-byte configuration space. The registers within this range are used to identify the device, control PCI bus functions and provide PCI bus status. The shaded areas indicate registers that are implemented in the PCI Interface Core. Unused registers produce a zero when read and ignore a write operation.

| Addr | Byte | | | |
|---|---|---|---|---|
| | 3 | 2 | 1 | 0 |
| 00h | Device ID | | Vendor ID | |
| 04h | Status Register | | Command Register | |
| 08h | Class Code | | | Revision ID |
| 0Ch | BIST | Header Type | Latency Timer | Cache Line Size |
| 10h | Base Address Register 0 | | | |
| 14h | Base Address Register 1 | | | |
| 18h | Base Address Register 2 | | | |
| 1Ch | Base Address Register 3 | | | |
| 20h | Base Address Register 4 | | | |
| 24h | Base Address Register 5 | | | |
| 28h | Card Bus CIS Pointer | | | |
| 2Ch | Subsystem ID | | Subsystem Vendor ID | |
| 30h | Expansion ROM Base Address Register | | | |
| 34h | Reserved | | | |
| 38h | Reserved | | | |
| 3Ch | Maximum Latency | Minimum Grant | Interrupt Pin | Interrupt Line |

**Table 2 – PCI Configuration Space**

**Vendor ID** – This is a 16-bit read-only register that identifies the manufacturer of the device. The value of this register is assigned by the PCI SIG.

**Device ID** – This is a 16-bit read-only register that identifies the device type.

**Command Register** – This is a 16-bit read/write register that provides basic control of the PCI function to respond to the PCI bus and/or access it. See table 3 for a detailed description.

| Bit | Name | Access | Description |
|---|---|---|---|
| 0 | IO_EN | Read/write | I/O access enable. Controls a device's response to I/O accesses. When high, it lets the device respond to the PCI bus I/O accesses as a target. |
| 1 | MEM_EN | Read/write | Memory access enable. Controls a device's response to Memory space accesses. When high, it lets the device respond to the PCI bus memory accesses as a target. |
| 2 | MASTER_EN | Read/write | Master enable |
| 3 | SPEC_CYC | Read/write | Special Cycle Enable. Controls a device's action on the Special Cycle operation. The current version of the PCI Interface Core doesn't support this feature, however it is possible to add Special Cycle support by the user. |
| 4 | MWI_EN | Read/write | Memory write and invalidate enable. |
| 5 | VGA_PS | Read | VGA Palette Snoop. Currently unused. The bit is hard-wired to zero. |
| 6 | PERR_EN | Read/write | Parity error enable. When high, it enables the function to report parity errors via the `perrn_p` output. |
| 7 | STEP_EN | Read | Stepping Enable. The current version of the PCI Core doesn't support address/data stepping. |
| 8 | SERR_EN | Read/write | System error enable. When high, it allows the function to report address parity errors via the `serrn_p` output. |
| 15:9 | Reserved | - - | Reserved bits are hard-wired to zero. |

**Table 3 – Command Register Format**

**Status Register** – This is a 16-bit register that provides the status of bus-related events. Read transactions from the status register behave normally. However, write transactions are different from typical write transactions because bits in the status register can be cleared but not set. A bit in the status register is cleared by writing a logic one to that bit. For a detailed description see table 4.

**Revision ID Register** – This is an 8-bit read-only register that identifies the revision number of the device. The value of this register is assigned by the manufacturer (designer). Revision ID is defined in pci_m64_params.v by the constant *REV_ID.*

**Class Code Register –** This is a 24-bit read-only register divided into three sub-registers: base class, sub-class, and programming interface. Class code is defined in pci_m64_params.v by the constant *CLASS_ID.*

**Cache Line Size Register –** This specifies the system cache line size in DWORDS. This read/write register is initialized by a system software at power-up.

**Latency Timer Register –** This is an 8-bit register. The register defines the maximum amount of time, in PCI bus clock cycles, that the PCI function can retain ownership of the PCI bus.

| Bit | Name | Access | Description |
|---|---|---|---|
| 3:0 | - | - | Reserved. |
| 4 | CAP_EN | Read | Read Capabilities list enable. When set, this bit enables the capabilities list pointer register at offset 34h. |
| 5 | PCI_66M | Read | Read PCI 66-MHz capable. When set, it indicates that the PCI device is capable of running at 66 MHz. |
| 6 | reserved | Read | The bit is hardwired to zero. |
| 7 | FBA_CAP | Read | Fast back-to-back capable. The bit is hardwired to zero. |
| 8 | MDPERR_DET | Read/write | Master Data Parity Error. When high, it indicates that during a read transaction the function asserted the PERRn output as a master device, or that during a write transaction the PERRn output was asserted as a target device. |
| 10:9 | DEVSEL_TIM | Read | Device select timing. The devsel_tim bits indicate target access timing of the function via the DEVSELn output. *The PCI-M64AHB Core is designed to be medium speed target device ("01"b)* |
| 11 | TABORT_SIG | Read/write | Signaled target abort. This bit is set when a local peripheral device terminates a transaction. |
| 12 | TABORT_DET | Read/write | Detected Target abort. When high, it indicates that the function in master mode has detected a target abort from the current target device |
| 13 | MABORT_SIG | Read/write | Master abort. When high, *MABORT_SIG* indicates that the function in master mode has terminated the current transaction with a master abort. |
| 14 | SERR_SIG | Read/write | Signaled system error. When high, *SERR_SIG* indicates that the function drove the SERR# output active, i.e., an address phase parity error has occurred. The function signals a system error only if an address phase parity error was detected and *SERR_EN* was set. |
| 15 | PERR_DET | Read/write | Detected parity error. When high, *PERR_DET* indicates that the function detected either an address or data parity error. Even if parity error reporting is disabled (via perr_ena), the function sets the *PERR_DET* bit. |

**Table 4 – Status Register Format**

**Header Type Register –** This is an 8-bit read-only register that identifies the PCI function as a single-function device. The PCI-M64AHB Core supports only header type 00.

**Base Address Registers** - The PCI function supports up to six BARs. Each base address register (BARn) has identical attributes. The BAR is formatted per the PCI Local Bus Specification, Revision 2.2. Bit 0 of each BAR is read only and it is used to indicate whether the reserved address space is memory or I/O. BARs that map to memory space must hardwire bit 0 to 0, and BARs that map to I/O space must hardwire bit 0 to 1. The format of the BAR changes depending on the value of bit 0.

**Interrupt Line Register** – This is an 8-bit register that defines to which system interrupt request line the INTA# output is routed. The Interrupt Line Register value is defined in pci_m64_params.v by the constant *INT_LINE.*

**Interrupt Pin Register** – This is an 8-bit read-only register that defines the PCI function PCI bus interrupt request line to be INTAn. The only allowed values are 00 (no interrupt line used) and 01 (INTA# line used). The Interrupt Pin Register value is defined in pci_m64_params.v by the constant *INT_PIN.*

**Minimum Grant Register** – This is an 8-bit read-only register that defines the length of time the function would like to retain mastership of the PCI bus. The minimum Grant value is defined in pci_m64_params.v by the constant *MIN_GNT.*

**Maximum Latency Register** – This is an 8-bit read-only register that defines the frequency in which the function would like to gain access to the PCI bus. Maximum Latency value is defined in pci_m64_params.v by the constant *MAX_LAT*.

## 3.3 Control Registers

Control registers are mapped in a 512-byte memory space that can be accessed both from the PCI bus and the AHB bus. The registers allow core function configuration, DMA transfer initiation, message passing through mailboxes and interrupt control.

Reserved registers are not implemented and read as zeros.

| Addr | Byte | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 000h | reserved | | | | PCIAHB_OFFSET | | | |
| 008h | reserved | | | | PCIAHB_ERROR | | | |
| 010h | reserved | | | | PCIAHB_DISCARD | | | |
| 018h : 078h | reserved | | | | reserved | | | |
| 080h | reserved | | | | DMA_PCIADDR | | | |
| 088h | reserved | | | | DMA_AHBADDR | | | |
| 090h | reserved | | | | DMA_TXSIZE | | | |
| 098h | reserved | | | | DMA_CTRL | | | |
| 0A0h | reserved | | | | DMA_STATUS | | | |
| 0A8h : 0F8h | reserved | | | | reserved | | | |
| 100h | reserved | | | | INTSTATUS | | | |
| 108h | reserved | | | | PCI_INTMASK | | | |
| 110h | reserved | | | | AHB_INTMASK | | | |
| 118h : 178h | reserved | | | | reserved | | | |
| 180h | PCI Mailbox 0 | | | | | | | |
| 188h | PCI Mailbox 1 | | | | | | | |
| 190h | PCI Mailbox 2 | | | | | | | |
| 198h | PCI Mailbox 3 | | | | | | | |
| 1A0h | AHB Mailbox 0 | | | | | | | |
| 1A8h | AHB Mailbox 1 | | | | | | | |
| 1B0h | AHB Mailbox 2 | | | | | | | |
| 1B8h | AHB Mailbox 3 | | | | | | | |
| 1C0h : 1F8h | reserved | | | | reserved | | | |

**Table 5 - Control registers address map**

# 4 Block Diagram

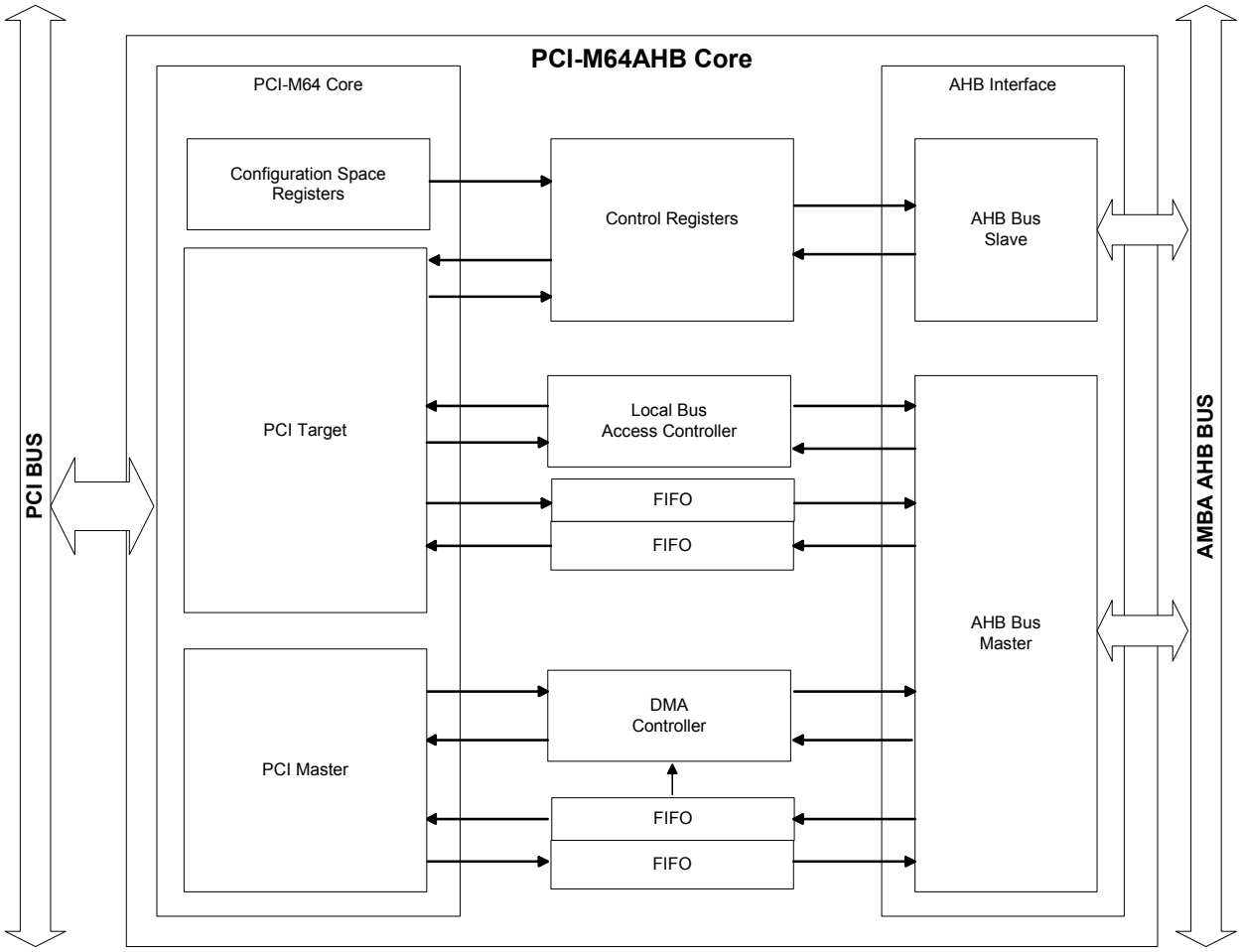


**Figure 2 – PCI-M64AHB Block Diagram**

# 5 Block Descriptions

As shown above and explained below, the PCI-M64AHB includes six major blocks: Parity Generator, Parity Checker, Configuration Space Registers, Command Register and Address Counter block, Target State Machine, and Master State Machine.

## 5.1 PCI-M64 Core

The PCI-M64 core controls access to the PCI bus. The PCI-M64 core can be divided to three major subsystems – PCI Target controller, PCI Master controller and Configuration Space register.

## 5.2 Target Access Controller

The target access controller handles transactions from the PCI bus to the AHB memory space initiated by a device on the PCI bus.

## 5.3 DMA Controller

The DMA Controller allows efficient high-speed data transfers between the PCI address space and the AHB address space.

## 5.4 Control Registers

The Control Registers control core functionality including DMA channel, PCI-AHB window, and interrupts. The control register block includes PCI and AHB mailboxes.

## 5.5 AHB Bus Slave

The AHB Bus Slave allows devices on the AHB bus to access core control registers.

## 5.6 AHB Bus Master

The AHB Bus Master performs data transfers on the AHB bus.

# 6 PCI-AHB Transactions

AHB memory space is mapped to the PCI memory space through Base Address Register 1 (BAR1).

PCI written data are stored in the AHB Write FIFO and then written to the AHB bus by the AHB Bus master. Maximum supported burst length is 15. After the 15$^{th}$ write the PCI transaction is disconnected. 8-bit and 16-bit transactions are disconnected after the first write.

PCI read transaction initiates AHB read transaction. The PCI transaction is terminated with target retry until the data is available. When a data prefetch is enabled, 16 DWORDs data are prefetched from the AHB bus and then forwarded to the PCI bus.

PCI transactions are terminated with target retry when another transfer is in progress. The PCI transaction initiator has to restart the transaction later.

PCI address is translated to AHB address using the PCI-AHB Offset register. The AHB address is the result of the PCI-AHB Offset register addition with address offset in the BAR1 space.

AHB Address = PCIAHB_OFFSET + (PCI Address – BAR1 Address)

There are three types of transfer errors during the PCI-AHB transaction:

- write post error – PCI device wrote data to the PCI-M64AHB core and the AHB master detected error when forwarded the data to the AHB bus.
- read prefetch error – AHB master detected an error when read data from the AHB bus.
- discard error – PCI device initiated reading of the AHB data. The transaction was retried and the core prefetched data from the AHB bus. PCI device did not repeat the read access within the discard limit time. The data was discarded to prevent deadlock and allow other PCI devices access to AHB memory space.

The transfer errors are indicated in a PCI-AHB Error register and can generate an interrupt. For more details about the interrupts see chapter 9.

## 6.1 PCI-AHB Offset Register - PCIAHB_OFFSET (offset = 0x00)

The PCI-AHB offset register defines the beginning of the PCI-AHB window. The offset value is a 32-bit number where bits 0 and 1 are zeros. Register bit 0 is used for AHB read prefetch enabling.

| Bits | Name | Access | Description |
|---|---|---|---|
| 0 | PFENA | R/W | AHB Space read prefetch enable |
| 1 | Reserved | RO | Reserved bits are read as zero |
| 31:2 | AHB_OFFSET | R/W | AHB Space offset address |
| 63:32 | Reserved | RO | Reserved bits are read as zero |

**Table 6 - PCI-AHB Offset Register**

## 6.2 PCI-AHB Error Register - PCIAHB_ERROR (offset = 0x08)

| Bits | Name | Access | Description |
|---|---|---|---|
| 0 | T64POSTERR | R/C | PCI-AHB post error |
| 1 | T64FETCHERR | R/C | PCI-AHB fetch error |
| 2 | T64DISCARD | R/C | PCI-AHB read data discarded |
| 63:3 | Reserved | RO | Reserved bits are read as zero |

**Table 7 - PCI-AHB Error Register**

## 6.3 PCI-AHB Discard Register – PCIAHB_DISCARD (offset = 0x10)

PCI read access initiate AHB data read, while the PCI read is retried. If the read transaction is not retried within DLIMIT cycles from the moment, when data is ready for PCI transfer, data is discarded. The data discard timeout is indicated in the PCI-AHB Error register and can issue an interrupt.

| Bits | Name | Access | Description |
|---|---|---|---|
| 7:0 | DLIMIT | R/W | PCI-AHB Discard limit |
| 63:8 | Reserved | RO | Reserved bits are read as zero |

**Table 8 - PCI-AHB Discard Register**

# 7 DMA Controller

Features:

- Both 64-bit and 32-bit transaction support
- up to 16MB transfer length
- Interrupt generation on transfer completion or error event

The DMA controller consists of the DMA Control, Read FIFO, Write FIFO and DMA Registers . The PCI Address Pointer and the AHB Address Pointers contain addresses from which the next data transfer will start. The pointer is automatically incremented after a successful read/write. For 64-bit data transfers the address must be a quad-word aligned.

The DMA Transfer Size Register contains the number of Bytes to be read/written. The counter is automatically decremented after a successful read/write. The lowest two bits are always zero. For 64-bit data transfers the number of bytes must be a quad-word aligned.

To be able to transfer at the maximum speed, a 64-bit data should be transferred. The driver has to enable 64-bit data transfers in the DMA Control register in a TX64_ENA bit.

A successful transfer or a transfer error is indicated in the DMA Status register and can generate an interrupt. See chapter 7.1 and chapter 9 for more details.

## 7.1 DMA Error States

DMA transfer errors on the PCI bus could be caused by:

- Transfer aborted by the PCI master – there is no target responding to the master's request on the PCI bus. The DMA transfer is aborted.
- Transfer aborted by the PCI target – the target detected an error during access to its space. The DMA transfer is aborted upon the PCI target abort event detection.
- Parity Error – a parity error is detected by the PCI-M64 core. The parity error does not interrupt the DMA transfer.

Errors are signaled given by the PCI_ERR bit in the DMA Status register. The cause of the PCI errors is indicated in the PCI Status register, which must be accessed in order to clear the status bits (see Section 3.2).

A 64-bit access failure is detected when the core initiates a 64-bit PCI transaction and the target device claims 32-bit-only response. The 64-bit access failure is not a fatal error and does not abort a DMA transfer. The 64-bit access failure is indicated in the Status register by the TX_64F flag bit. This bit must be cleared by control software before a new 64-bit DMA transfer is initiated, otherwise the transaction will proceed as a 32-bit wide only.

## 7.2 DMA Registers

There are three types of access rights to any control/status register:

RO – Read Only – writing has no affect on the register contents.

WO – Write Only – reading of RO bits returns ones.

R/W – both Read and Write access allowed.

R/C - Read or Clear access. Clear is done by writing a logic one to the register's bit.

### 7.2.1 DMA PCI Pointer – DMA_PCIPTR (offset = 0x80)

The DMA PCI Pointer should be initialized by the physical address of the source buffer. The pointer is automatically incremented after a successful transfer and points to the new beginning of a DMA transfer on the PCI bus. The address must be a quad-word aligned for 64-bit data transaction.

| Bits | Name | Access | Description |
|---|---|---|---|
| 31:0 | DMA_PCIPTR | R/W | Pointer to PCI data buffer |
| 63:32 | N/A | RO | Reserved bits are read as zero |

**Table 9 - DMA PCI Pointer Register description**

### 7.2.2 DMA AHB Pointer – DMA_AHBPTR (offset = 0x88)

The DMA AHB Pointer should be initialized by the AHB address of the source/destination buffer. The pointer is automatically incremented after a successful transfer and points to the new beginning of a DMA transfer on the AHB bus.

| Bits | Name | Access | Description |
|---|---|---|---|
| 31:0 | DMA_AHBPTR | R/W | Pointer to AHB data buffer |
| 63:32 | N/A | RO | Reserved bits are read as zero |

**Table 10 - DMA AHB Pointer Register description**

### 7.2.3 DMA Transfer Counter – DMA_TXCNT (offset = 0x90)

The DMA Transfer Counter specifies a number of bytes to be transferred between PCI and AHB memories. For 64-bit data transfers the number of bytes must be a quad-word aligned. The counter is decremented after a successful read. The maximum number of transfers is 16MB.

| Bits | Name | Access | Description |
|---|---|---|---|
| 23:0 | DMA_TXCNT | R/W | Read data counter – counts bytes down to zero |
| 63:24 | Reserved | RO | Reserved bits are read as zero |

**Table 11 - DMA Transfer Counter Register description**

### 7.2.4 DMA Control Register – DMA_CTRL (offset = 0x98)

The DMA Control register enables DMA transfers. It also controls the interrupt generation and allows emptying the FIFOs if needed.

| Bits | Name | Access | Description |
|---|---|---|---|
| 3:0 | DMA_INS | R/W | DMA Instruction |
| 4 | DMA_TXENA | R/W | Enable DMA transfer |
| 5 | DMA_TX64 | R/W | 64-bit PCI Transfer Enable |
| 6 | DMA_FLSH | R/W | Flush DMA FIFOs |
| 63:7 | reserved | RO | Reserved bits are read as zero |

**Table 12 - DMA Control Register description**

Supported PCI commands:

- I/O Read "0010" (0x02)
- Memory Read "0110" (0x06)
- Memory Read Line "1110" (0x0E)
- Memory Read Multiple "1100" (0x0C)
- I/O Write "0011" (0x0003)
- Memory Write "0111" (0x007)

### 7.2.5 DMA Status Register – DMA_STATUS (offset = 0xA0)

| Bits | Name | Access | Description |
|---|---|---|---|
| 0 | RD_TC | RO | DMA Read Transfer Completed |
| 1 | PCI_ERR | R/C | PCI Error detected |
| 2 | AHB_ERR | R/C | AHB Error detected |
| 3 | TX_64F | R/C | 64-bit Access failed - 32-bit target response |
| 63:4 | reserved | RO | Reserved bits are read as zero |

**Table 13 - DMA Status Register description**

# 8 Mailboxes (offset = 0x180-0x1BF)

Mailboxes are useful for message passing and interrupt generation. The PCI-M64AHB implements a set of eight 64-bit mailboxes, four for each direction.

The mailbox data write changes mailbox contents and the mailbox full status bit is asserted. The mailbox full status bit is cleared when the mailbox is read.

Mailbox registers are at offset range 0x180-0x1BF.

The first four mailboxes (PCI mailboxes) can be written from the AHB bus and read from the PCI bus.

The second group of mailboxes can be written from the PCI bus and read from the AHB bus.

# 9 Interrupts

The PCI-M64AHB core can generate interrupts to both PCI bus and AHB bus sides. The interrupt generation is controlled by the PCI interrupt mask register and the AHB interrupt mask register.

The PCI-M64AHB core uses only the intan_p signal for PCI interrupts. All the internal subsystems use the same interrupt line which is adjusted by the BIOS during startup. The value of the interrupt line is stored in the Interrupt Line register in the card PCI Configuration Space.

The PCI-M64AHB core has one interrupt line (hint) for AHB bus system interrupt signaling. The interrupt assertion is controlled by an AHB Interrupt Mask register.

## 9.1 Interrupt Status Register - INTSTATUS (offset = 0x100)

The Interrupt Status register is a read-only register, which indicates status of all interrupt sources. Status bit clearing has to be done in the corresponding subsystem status register.

| Bits | Name | Access | Description |
|---|---|---|---|
| 0 | T64POSTERR | RO | PCI-AHB post error |
| 1 | T64FETCHERR | RO | PCI-AHB fetch error |
| 2 | T64DISCARD | RO | PCI-AHB read data discarded |
| 7:3 | Reserved | RO | Reserved bits are read as zero |
| 8 | DMARDTC | RO | DMA transfer completed |
| 9 | DMAPCIERR | RO | DMA PCI bus transfer error |
| 10 | DMAAHBERR | RO | DMA AHB bus transfer error |
| 15:11 | Reserved | RO | Reserved bits are read as zero |
| 16 | PCIMBOXFULL_0 | RO | PCI Mailbox 0 full |
| 17 | PCIMBOXFULL_1 | RO | PCI Mailbox 1 full |
| 18 | PCIMBOXFULL_2 | RO | PCI Mailbox 2 full |
| 19 | PCIMBOXFULL_3 | RO | PCI Mailbox 3 full |
| 20 | AHBMBOXFULL_0 | RO | AHB Mailbox 0 full |
| 21 | AHBMBOXFULL_1 | RO | AHB Mailbox 1 full |
| 22 | AHBMBOXFULL_2 | RO | AHB Mailbox 2 full |
| 23 | AHBMBOXFULL_3 | RO | AHB Mailbox 3 full |
| 31:24 | Reserved | RO | Reserved bits are read as zero |

**Table 14 - DMA Status Register description**

## 9.2 PCI Interrupt Mask Register – PCI_INTMASK (offset = 0x108)

The PCI Interrupt Mask register enables signaling of successful DMA transfer, data transfer errors and PCI mailbox writes to the PCI bus by INTA# assertion. The mask register bits correspond to the status bits in the Interrupt Status register.

| Bits | Name | Access | Description |
|---|---|---|---|
| 0 | T64POSTERR | R/W | PCI-AHB post error interrupt mask |
| 1 | T64FETCHERR | R/W | PCI-AHB fetch error interrupt mask |
| 2 | T64DISCARD | R/W | PCI-AHB read data discarded interrupt mask |
| 7:3 | Reserved | RO | Reserved bits are read as zero interrupt mask |
| 8 | DMARDTC | R/W | DMA transfer completed interrupt mask |
| 9 | DMAPCIERR | R/W | DMA PCI bus transfer error interrupt mask |
| 10 | DMAAHBERR | R/W | DMA AHB bus transfer error interrupt mask |
| 15:11 | Reserved | RO | Reserved bits are read as zero |
| 16 | PCIMBOXFULL_0 | R/W | PCI Mailbox 0 full interrupt mask |
| 17 | PCIMBOXFULL_1 | R/W | PCI Mailbox 1 full interrupt mask |
| 18 | PCIMBOXFULL_2 | R/W | PCI Mailbox 2 full interrupt mask |
| 19 | PCIMBOXFULL_3 | R/W | PCI Mailbox 3 full interrupt mask |
| 31:20 | Reserved | RO | Reserved bits are read as zero |

**Table 15 – PCI Interrupt Mask Register description**

## 9.3 AHB Interrupt Mask Register – AHB_INTMASK (offset = 0x110)

The AHB Interrupt Mask register enables signaling of successful DMA transfer, data transfer errors and AHB mailbox writes to the AHB bus by hint output assertion. The mask register bits correspond to the status bits in the Interrupt Status register.

| Bits | Name | Access | Description |
|---|---|---|---|
| 0 | T64POSTERR | R/W | PCI-AHB post error interrupt mask |
| 1 | T64FETCHERR | R/W | PCI-AHB fetch error interrupt mask |
| 2 | T64DISCARD | R/W | PCI-AHB read data discarded interrupt mask |
| 7:3 | Reserved | RO | Reserved bits are read as zero interrupt mask |
| 8 | DMARDTC | R/W | DMA transfer completed interrupt mask |
| 9 | DMAPCIERR | R/W | DMA PCI bus transfer error interrupt mask |
| 10 | DMAAHBERR | R/W | DMA AHB bus transfer error interrupt mask |
| 19:11 | Reserved | RO | Reserved bits are read as zero |
| 20 | AHBMBOXFULL_0 | R/W | AHB Mailbox 0 full interrupt mask |
| 21 | AHBMBOXFULL_1 | R/W | AHB Mailbox 1 full interrupt mask |
| 22 | AHBMBOXFULL_2 | R/W | AHB Mailbox 2 full interrupt mask |
| 23 | AHBMBOXFULL_3 | R/W | AHB Mailbox 3 full interrupt mask |
| 31:24 | Reserved | RO | Reserved bits are read as zero |

**Table 16 - AHB Interrupt Mask Register description**

# 10 PCI Electrical Specification

All the electrical characteristics and constraints are described in the PCI Local Bus Specification Rev. 2.2, chapter 4. The user is strongly advised to read the specification especially for ASIC implementations.

# 11 PCI Timing Constraints

| Symbol | Parameter | Min | Max | Units |
|---|---|---|---|---|
| $T_{cyc}$ | CLK cycle time | 15 | ∞ | ns |
| $T_{high}$ | CLK High time | 6 | | ns |
| $T_{low}$ | CLK Low time | 6 | | ns |
| $T_{val}$ | CLK to signal valid delay – bussed signals | 2 | 6 | ns |
| $T_{val(ptp)}$ | CLK to signal valid delay – point to point | 2 | 6 | ns |
| $T_{on}$ | Float to active delay | 2 | | ns |
| $T_{off}$ | Active to float delay | | 14 | ns |
| $T_{su}$ | Input setup time to CLK – bussed signals | | 3 | ns |
| $T_{h}$ | Input hold time from CLK | | 0 | ns |
| $T_{rst-off}$ | Reset active to output float delay | | 40 | ns |

**Table 17 - PCI Timing Constraints**

# 12 Supplied Files

.\doc\ - documentation
    cast_pci-m64ahb_ug.pdf - PCI-M64AHB User's Guide
.\scripts\ - synthesis and simulation scripts
    compile_mti - ModelSim simulation script
    wave_mti.do - ModelSim Waveform script
    exemplar_virtex2.tcl - Leonardo script for Virtex-II
    synopsys.scr - Design Compiler sample synthesis script

.\src\ - source files

\m64core\ - PCI-M64 Core source files

| | |
|---|---|
| pci_barreg.v (vhd) | - Base Address Register (BAR) |
| pci_cead.v (vhd) | - AD output FFs clock enable |
| pci_cfgebar.v (vhd) | - Expansion ROM BAR |
| pci_cfgspace.v (vhd) | - Configuration space top level entity |
| pci_gclkbuf.v (vhd) | - PCI clock buffer |
| pci_genack64.v (vhd) | - ACK64# generator |
| pci_gendevsel.v (vhd) | - DEVSEL# generator |
| pci_genframe.v (vhd) | - FRAME# generator |
| pci_genirdy.v (vhd) | - IRDY# generator |
| pci_genotad.v (vhd) | - AD lines tri-state control |
| pci_genotcbe.v (vhd) | - C/BE lines tri-state control |
| pci_genpar64.v (vhd) | - parity generator |
| pci_genreq64.v (vhd) | - REQ64# generator |
| pci_genstop.v (vhd) | - STOP# generator |
| pci_gentrdy.v (vhd) | - TRDY# generator |
| pci_chkpar64.v (vhd) | - parity checker |
| pci_ibuf.v (vhd) | - PCI input buffer |
| pci_iobuf.v (vhd) | - PCI bidirectional buffer |
| pci_m64_params.v (vhd) | - core parameter constants (template) |
| pci_m64core.v (vhd) | - Core top level entity |
| pci_mtfsm64.v (vhd) | - Core control FSM |
| pci_obuf.v (vhd) | - PCI output buffer |
| pci_obuft.v (vhd) | - PCI output tri-state buffer |
| pci_obufoc.v (vhd) | - PCI output buffer - open collector |

\ahb\ - AHB Interface source files

| | |
|---|---|
| dma64_fiforam.v (vhd) | - RAM used in FIFOs |
| dma64rdfifo.v (vhd) | - Read FIFO |
| dma64wrfifo.v (vhd) | - Write FIFO |
| dma64regs.v (vhd) | - DMA registers |
| dma64txctrl.v (vhd) | - DMA control |
| dma64ahb.v (vhd) | - DMA core top level entity |
| ahbmaster.v (vhd) | - AHB Master |
| ahbslave.v (vhd) | - AHB slave |
| fifo_async.v (vhd) | - Asynchronous FIFO |
| fifosram.v (vhd) | - Asynchronous dual-port RAM |
| intctrl.v (vhd) | - Interrupt controller |
| mailbox.v (vhd) | - Mailbox |
| mailboxblk.v (vhd) | - Mailbox set |
| mt64ahb.v (vhd) | - AHB bridge module |
| pci_m64_params.v (vhd) | - Core parameters |
| pcim64ahb.v (vhd) | - PCI-M64AHB core |

| | |
|---|---|
| pcim64ahbw.v (vhd) | - PCI-M64AHB core I/O wrapper |
| psync.v (vhd) | - Asynchronous clock domain pulse sync |
| t64ahb.v (vhd) | - PCI-AHB target controller |

.\tb\ - testbench files

| | |
|---|---|
| pci64_params.v (vhd) | - PCI constants |
| pci64_busmonitor.v (vhd) | - Bus monitor |
| pci_arbiter_model.v (vhd) | - PCI bus arbiter |
| pci_master64model.v (vhd) | - PCI master simulation model |
| pci_target64model.v (vhd) | - PCI target simulation model |
| ahbslaveram_model.v (vhd) | - AHB Slave RAM model |
| pcim64ahbw_tb.v (vhd) | - Testbench |
| ram_burst.vec | - Simulation vector file |
| t64_t1.vec | - Simulation vector file |
| t64_t2.vec | - Simulation vector file |
| pci_m64.sim.log | - Simulation log file |
| bus_monitor.log | - Bus monitor logfile |
| t64_t1.log | - Simulation log file |
| t64_t2.log | - Simulation log file |

# 13 Design Guidelines

The PCI-M64AHB Interface core has an application design template prepared. There is a top-level module wrapper (pcim64ahbw.v*) that consists of the PCI-M64AHB Interface Core (PCI-M64AHB) module and the PCI I/O buffers.

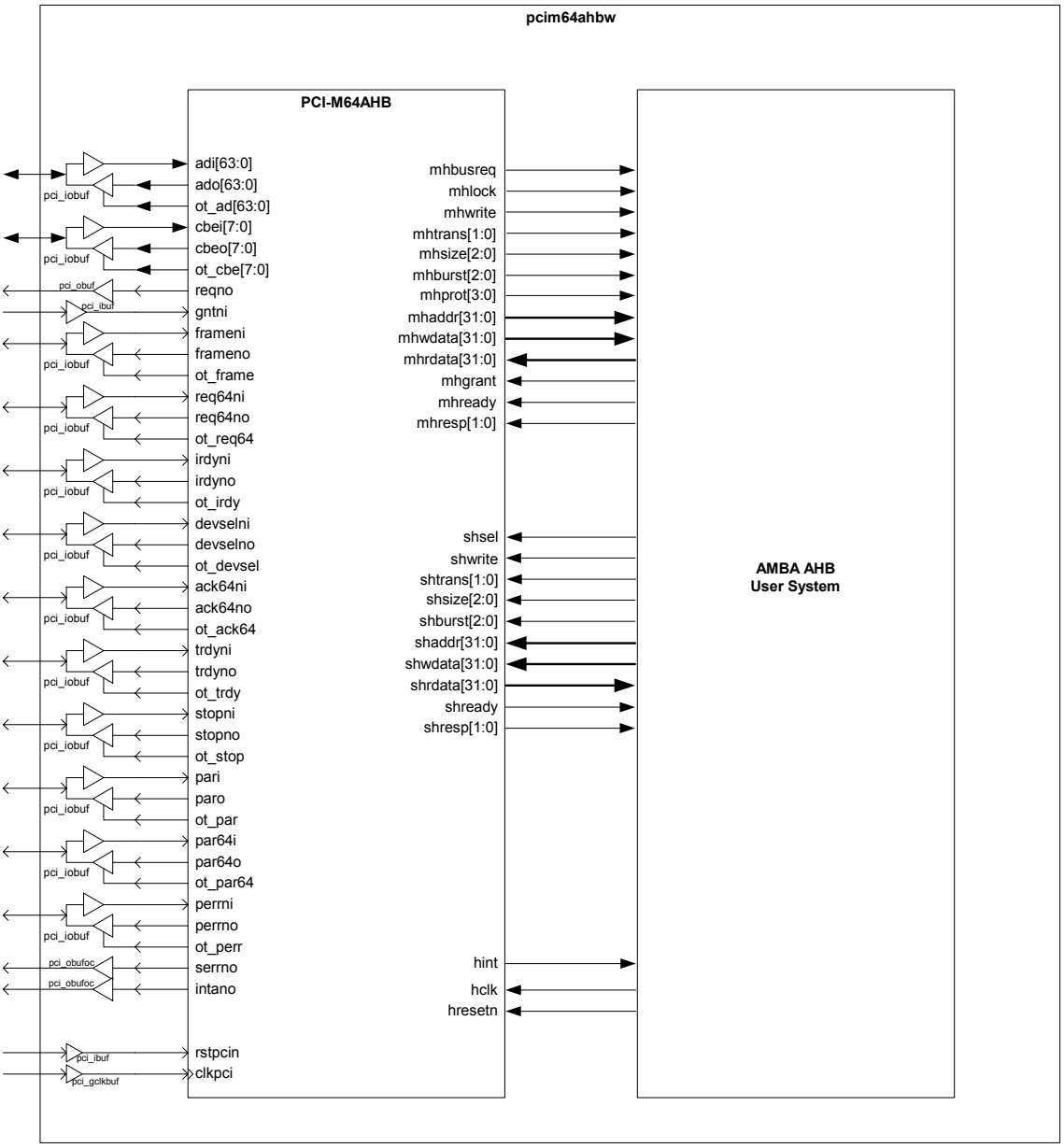


**Figure 3 - PCI-M64AHB design hierarchy**

## 13.1 Configuration space setup

All the adjustable parameters for the PCI Configuration space setup are stored in the pci_m64_params.v file.

**Vendor ID** – Is assigned by the PCI SIG. Example uses number 0x0ABCD, which is currently free. User must ask PCISIG for a valid Vendor ID assignment to be PCI specification compliant.

```
parameter[15:0] vendor_id = 16'b1010101111001101; //Vendor ID
```

**Device ID** – User defined number

```
parameter[15:0] device_id = 16'b0110010001100110; //Device ID
```

**Class ID** – The recommended ClassID for the PCI-M64AHB core is "068000".

```
parameter[23:0] class_id = {8'b00000110, 8'b10000000, 8'b00000000}; //Device Class
```

**Revision ID** – User defined number

```
-- Revision ID                                  76543210
constant REV_ID: std_logic_vector(7 downto 0):="00010000"; -- Revision ID = 1.0
```

**Base Address Register Setup**

Each Base Address Register can be adjusted to decode I/O or Memory accesses. Each BAR register is enabled by the BARx_PRESENT constant. The type of address space decoding and address range is defined by the BARx_MAP constant. (see PCI Spec. Rev. 2.2 Section 6.2.5.1 on page 201).

Base Address registers asking for I/O space should allocate 256 bytes or less.

For the PCI-M64AHB core it is recommended to map AHB space to the PCI Memory space. The mapping is done by the Base Address Register 1 (BAR1).

Settings of the BAR0 and BAR2 to BAR5 should not be changed. BAR0 is used for control register access and BAR2 to BAR5 are unused.

Example:

We need BAR1 to decode 16MB Memory space. This block can be positioned anywhere in the 32-bit address space and is not prefetchable. Then we have to setup the following values in the pci_m64_params.v file:

```
//BAR1 parameters

parameter bar1_present = 1'b1;

//                            33222222222211111111111

//                            10987654321098765432109876543210

parameter[31:0] bar1_map = 32'b11111111000000000000000000000000;

parameter bar1_dwidth = 8;  //16MB space
```

**Subsystem Vendor ID** – Has the same rules as Vendor ID.

```
-- Subsystem Vendor ID                                FEDCBA9876543210
constant SUBVENDOR_ID : std_logic_vector(15 downto 0):="1010101111001101"; -- Vendor ID =
0x0ABCD (void)
```

**Subsystem Device ID** – Has the same rules as Device ID.

```
-- Subsystem Device ID                                FEDCBA9876543210
constant SUBDEVICE_ID : std_logic_vector(15 downto 0):="0000000100000000"; -- Device ID =
x0100
```

**Maximum Latency** - Defines a period (in 250ns steps) of how often the device will require to perform master transactions (for more details see PCI Spec. Rev. 2.2 page 200). A value of 32 means that device will need to transfer data every 8 µs.

```
-- Maximum Latency                                76543210
constant MAX_LAT: std_logic_vector(7 downto 0):="00100000"; --
```

**Minimum Grant** - Defines a burst length needs in 250ns steps. If we have a device with 128 bytes deep buffer (32x32b) and we need to burst all the buffer data then the Minimum Grant value will be 4 (for more details see PCI Spec. Rev. 2.2 page 200)

```
-- Minimum Grant                                  76543210
constant MIN_GNT: std_logic_vector(7 downto 0):="00000100"; --
```

**Interrupt Pin** – Defines which interrupt pin the device uses. A value of 1 corresponds to INTA# pin, value of 0 means no interrupt pin used.

```
-- Interrupt Pin                                  76543210
constant INT_PIN: std_logic_vector(7 downto 0):="00000001";
```

# 14 Simulation Notes

The core deliverables contain simulation scripts for the ModelSim simulator. It is recommended that a directory *sim* is created parallel to the *src* directory for all simulation runs. Vector files have to be copied to the *sim* directory prior to simulating.

# 15 Core Modifications

The PCI-M64AHB core can be customized to include:

- additional DMA channels
- additional PCI-AHB windows
- 64-bit PCI addressing support capability

Please contact CAST for any required modifications.

# 16 Support

Every effort has been made to ensure that this core functions correctly. If a problem is encountered please contact:

CAST, Inc.
11 Stonewall Court
Woodcliff Lake, NJ 07677 USA
Technical Support Hotline: +1-201-391-8300
Fax: +1-201-391-8694
E-mail: support@cast-inc.com
URL: www.cast-inc.com

# 17 Related Information

[1] PCI Local Bus Specification Rev. 2.2, PCISIG, www.pcisig.com

[2] Solari E., Wilse G.: PCI Hardware and Software Architecture & Design, 4th edition, Annabooks, 1998